

Beyond Lecture and Lab: Deliberate Gap Learning as an AI-Era Homework Framework

Brian S. Mitchell
College of Engineering and Computing
Drexel University
Philadelphia, PA USA

Abstract—The rapid proliferation of large language models has disrupted the effort economics of undergraduate computing homework. When assignments are easily automated, students are incentivized toward shortcuts rather than deep engagement, and prior AI-integration approaches in computing education have focused on detection, restriction, or tool fluency rather than homework redesign. This paper introduces Deliberate Gap Learning (DGL), an open, tool-agnostic framework that preserves educational integrity in courses where generative AI can effortlessly solve conventional computing tasks.

DGL is grounded in inquiry-based learning: lecture establishes conceptual foundations, labs develop procedural competence, and homework bridges an intentional gap through a four-part critical LLM dialogue anchored to the most recent lab. The framework targets topics deliberately left uncovered in lecture, requiring students to use AI as a learning partner rather than a task-completion engine. A process-focused rubric assesses investigative quality rather than answer correctness, making shortcut strategies structurally unattractive by ensuring that gaming the assignment requires effort comparable to authentic engagement.

Across seven assignments in an undergraduate systems architecture course ($N = 105$; 538 survey responses), deep AI engagement was sustained at 87–96% with no drift toward low-effort behaviors over the semester. Pooled Hake normalized gain was $\bar{g} = 0.46$, with higher gains for processor architecture units ($\bar{g} = 0.54$) than hardware design ($\bar{g} = 0.34$). Perceived learning value declined steadily across the semester, revealing constraints around cognitive accessibility and format fatigue that warrant further investigation.

I. INTRODUCTION

For decades, undergraduate computer science homework has operated on the foundational assumption that learning happens through the struggle of working toward a solution. In lab-intensive courses, this model was already under pressure before AI arrived. When lectures establish conceptual foundations and hands-on labs serve as the primary vehicle for practice and skill development, the traditional problem-set homework loses its natural role. Students are already learning by building, which raises a question most course designs leave unanswered: if labs have taken over applied practice, what is homework actually for?

The emergence of capable large language model (LLM) tools such as ChatGPT, Claude, and Gemini has fundamentally changed the effort economics of traditional homework. A competent modern LLM can produce correct, well-explained solutions to most undergraduate-level programming problems, algorithm questions, and conceptual exercises quickly. Because LLM outputs vary with context and phrasing, no two

responses are identical, greatly complicating attribution the way a direct source match once could when suspecting academic integrity violations. It has become a common observation among many instructors that grade distributions on conventional assignments have shifted upward, while these gains often correlate poorly with performance on controlled assessments where AI assistance is constrained.

This is not primarily a story about academic integrity. It is a story about incentives. In an environment where AI can provide correct answers with minimal effort, a student who chooses to work through a problem genuinely (investing the time to understand rather than just to produce) is, by the logic of grades-as-incentives, behaving suboptimally. This creates a learning environment that rewards surface-level engagement and punishes depth. The problem is not student character; it is assignment design that has not yet caught up with a world where AI tool fluency is an expected professional skill and a submitted assignment no longer signals what a student knows, only what they chose not to delegate.

The response of much of the field has been defensive: AI-detection tools, proctored environments, oral defenses, or outright prohibition of AI assistance. These approaches have real costs. AI detection tools produce significant false positives and false negatives [1], [2]. Proctored environments do not represent the conditions under which students will actually work after graduation. And prohibition trains students to work without tools they will be expected to use professionally [3].

The same capability that makes AI tools effective at solving conventional homework problems makes them potentially powerful as learning partners. Used as a learning partner rather than an answer-generating machine, an LLM offers on-demand tutoring that scales to an entire class. The pedagogical literature on inquiry-based learning [4] suggests that constructing understanding through active questioning produces more durable learning than simply receiving correct answers. The catch is assignment design: it must *require* students to *question*, *investigate*, and *reflect*, rather than simply allow AI to be used as a shortcut.

The key insight driving this work is that the challenge is not that AI can provide answers. The problem is that conventional homework only requires answers. If we redesign homework to require evidence of process (a documented inquiry log, a synthesis written from memory, a critical evaluation of AI claims, and a genuine connection to prior hands-on work), then submitting an AI-generated answer is no longer enough. This

alignment matters beyond the classroom: students entering the workforce are increasingly expected to use AI to amplify their own thinking rather than replace it [3], and an assignment model that rewards genuine AI-guided inquiry trains exactly that capacity.

Against this backdrop, a natural question is whether CS students, arguably more attuned to AI capabilities than students in most other disciplines and under explicit career pressure to demonstrate AI fluency, might be uniquely receptive to a homework format that finally *requires* rather than restricts AI use. We entered this study with exactly that hypothesis: that students who have spent their academic careers having AI policed or forbidden would embrace an assignment design that treats deep AI engagement as the goal. This paper reports what we found and examines why the gap between that expectation and the actual results is itself an important finding for the design of AI-integrated learning environments.

This paper introduces **Deliberate Gap Learning (DGL)**, a homework design framework built on a simple premise: we propose that a more educationally productive use of AI is not to answer questions, but to help students learn how to ask better ones. We describe the design principles underlying DGL and report a full deployment across seven assignments in an undergraduate systems architecture course ($N = 105$), presenting empirical findings from a longitudinal survey instrument administered alongside each assignment. The study represented a baseline characterization where outcomes confirmed some hypotheses, surfaced interesting unexpected findings, and motivated a set of follow-up investigations discussed later in this paper.

II. RELATED WORK

DGL sits at the intersection of five areas of prior work: foundational research on inquiry-based learning that motivates the pedagogical approach, empirical studies on AI tools in CS education that establish the problem context, the growing literature on academic integrity in the LLM era, emerging frameworks for AI literacy as a curricular goal, and methodological precedents for using self-reported learning gains as an outcome measure.

A. Inquiry-Based Learning (IBL)

The pedagogical foundation for DGL lies in a well-established body of evidence showing that structured self-directed inquiry produces more durable learning than direct instruction. Freeman et al. [5], in a meta-analysis of 225 STEM studies, found that active learning produced significantly higher exam scores and lower failure rates than traditional lecture. The Investigation Log and Synthesis requirements in DGL are direct instantiations of the problem-based learning tradition, in which the learning process itself rather than the answer is the primary outcome.

DGL diverges from classical IBL in one structural way: the AI replaces the instructor or peer group as the inquiry partner. This enables unlimited scaling but introduces a challenge human-guided inquiry does not face: the AI can be fluently wrong. DGL addresses this by requiring students to identify

and follow up on at least one imprecise or incorrect AI claim as a graded rubric component, making epistemic scrutiny an explicit expectation rather than an implicit one.

B. AI Tools in CS Education

The empirical case that large language models have fundamentally changed the landscape of CS homework is now well-documented. Finnie-Ansley et al. [6] demonstrated that OpenAI Codex could outperform a substantial portion of students on introductory programming exam problems. Becker et al. [7] extended this discussion to Copilot and other AI code-generation tools and framed the pedagogical challenge directly: programming assignments that have anchored CS education for decades can now be completed, correctly and fluently, with a well-placed prompt. Prather et al. [8], [9] found that even students who benefited from AI assistance often struggled to understand or verify what the tool produced, a pattern consistent with surface engagement rather than genuine learning.

C. Academic Integrity in the LLM Era

The dominant institutional response to AI-assisted assignment completion has been defensive: detection tools, proctored assessments, oral defenses, and prohibition. Detection tools produce substantial false positive rates and are demonstrably biased against non-native English speakers [2]. Perkins [10] argues that blanket prohibitions are the wrong institutional response, with clear acceptable-use policy a more productive alternative; Cotton et al. [11] make a similar case: framing AI use as a pure integrity problem treats a structural shift in how students work as a disciplinary matter rather than a pedagogical one.¹ Mollick and Mollick [12] contend that the more productive question is not how to prevent AI use but how to redesign assignments around it. DGL is one such redesign: assignments are structured so that AI use produces genuine learning rather than circumventing it.

D. Constructive Design Responses to AI in CS Education

Parallel to the defensive integrity discussion, a substantial body of work has shifted from asking how to prevent AI use to how to channel it pedagogically. Denny et al. [13] survey approaches including scaffolded AI tutors, course-level reorganization, and assignment-level redesign. Liffiton et al.’s CodeHelp [14] provides LLM-powered hints with explicit guardrails against direct solutions. Vadaparty et al. [15] introduce CS1-LLM, a full introductory programming course redesign that assumes Copilot access from day one. Kazemitabaar et al. [16] provide one of the few controlled empirical studies, finding that AI assistants improved code-authoring performance without harming code-modification skills. Kim and Hamdi [17] offer a recent empirical data

¹Notably, Cotton et al. disclose that the body of their own paper, from introduction through conclusion, was drafted directly by ChatGPT from a series of author-supplied prompts, with the authors’ contribution limited to prompting, structuring, and replacing several fabricated references the model had invented: a deliberate demonstration of the very capability their paper discusses.

TABLE I
COMPARISON OF APPROACHES TO INTEGRATING AI INTO COURSEWORK.

Approach	AI Role	Inquiry Graded	Homework Redesign	Custom Tooling
Prohibition / detection	Barred	No	No	Moderate
AI tutor (e.g., CodeHelp [14])	Hint provider	No	No	Yes
Full course redesign (e.g., CS1-LLM [15])	Copilot	Partial	Whole course	No
Unstructured AI use	Answer generator	No	No	No
LLM prompt session [17]	Prompt practice	No	No	No
DGL (this work)	Inquiry partner	Yes	Assignment level	No

point in a software engineering context. Embedding structured LLM prompt sessions alongside lectures produced a 4.19% average conceptual gain across 48 undergraduates, but gains in identifying incorrect LLM outputs reached only 3.67%, and improvements in understanding LLM limitations just 2.22%. The pattern suggests that structured access to AI does not reliably produce critical engagement without explicit graded incentives.

DGL complements rather than competes with these approaches: unlike CodeHelp (custom infrastructure) or CS1-LLM (full course redesign), DGL operates at the level of individual assignment design and requires no custom tooling.

Table I compares DGL to other approaches instructors have taken to integrate AI into coursework. The key differentiator is that DGL is the only approach in which *the inquiry process itself* is the graded artifact, making the AI interaction both required and unbyypassable without also completing the intended learning.

E. AI Literacy as an Educational Goal

A growing consensus has emerged that AI fluency (the ability to query effectively, interpret outputs critically, and synthesize across interactions) is a core professional competency. Long and Magerko [18] define AI literacy as competencies that allow individuals to critically evaluate AI technologies, communicate and collaborate with AI, and use AI across domains. Their framework identifies critical appraisal as central, and DGL targets this directly through its Critical Check and Investigation Log requirements. Ng et al. [19] find that existing AI literacy frameworks have emphasized applying AI in practice, with comparatively less attention to the higher-order work of critically evaluating and creating with AI. DGL operationalizes this underemphasized sense through its Critical Check requirement: students do not study how to evaluate AI outputs in the abstract, they practice identifying and following up on questionable AI claims under structured conditions with a graded incentive for genuine engagement.

F. Self-Reported Learning Gains and Survey Methodology

Self-reported measures of conceptual understanding are well-established in CS education research as a practical out-

come measure when matched pre/post content assessments are logistically prohibitive. This study adopts Hake’s normalized gain framework [20], with the standard low/medium/high thresholds treated as approximate reference points for ordering relative performance across assignments, not as validated criteria for self-reported instruments. Pre/post survey designs are subject to response-shift bias [21]; the retrospective pre-test, in which students rate prior knowledge at post-measurement after the learning experience has recalibrated their understanding, is the established mitigation [22] and is the approach used here.

III. THE DGL FRAMEWORK AND RESEARCH OBJECTIVES

A. Core Design Principle

The framework rests on a single core design principle: homework assignments should target topics that are *intentionally not covered in lecture*.

This is a departure from the conventional model, in which homework and assessments are drawn from content covered in class. DGL inverts this relationship. Lecture provides the foundational vocabulary and conceptual framework. Homework asks students to use that foundation to investigate something adjacent: a mechanism that extends what was taught, a design decision whose tradeoffs become visible only once the basics are understood, or a real-world implication the lab points toward but does not explain. Because the topic is genuinely not covered in lecture, the AI cannot simply recall it from course materials, and the process of investigation (following up on vague explanations, catching oversimplifications, synthesizing across threads) is precisely the work the assignment requires.

B. Assignment Structure

Each DGL assignment uses a four-section structure with a fixed 100-point rubric weighting:

- **Investigation Log (30 pts).** Students document 6–10 AI prompts with brief rationale for each follow-up, demonstrating genuine iterative engagement. A log showing a single exchange earns minimal credit; logs showing escalating specificity and pushback on vague answers earn full marks.
- **Synthesis (30 pts).** Students write a summary of what they learned entirely in their own words without referring back to the AI transcript. The underlying principle is that genuinely learning something through investigation means being able to explain it from scratch, much like answering a question in a technical interview. The synthesis is graded on accuracy and depth of understanding.
- **Critical Check (20 pts).** Students identify at least one AI claim that seemed unclear, oversimplified, or potentially incorrect and report how they followed up on it using their investigation log. The goal is to develop the habit of questioning rather than accepting AI output at face value, making AI skepticism an explicit graded requirement rather than an implicit expectation.
- **Connect It Back (20 pts).** Students draw an explicit connection between what they investigated and the lab work that preceded the assignment. DGL intentionally

TABLE II
DGL ASSIGNMENTS: LAB ANCHORS AND DELIBERATE GAPS FOR HOMEWORKS 1–7.

HW	Title	Lab Anchor	Deliberate Gap and Anticipated Learning Gain
1	The Hidden Cost of Intel/AMD: Complexity and Power	RISC-V fixed-width 32-bit instruction encoding; the load/store model	Why x86 processors are dramatically more complex and power-hungry than RISC-V: x86 instructions are translated into micro-operations before execution, and Dennard Scaling collapsed ~2004, ending free transistor-speed improvements.
2	The Secret Speed of Sequential Code: Cache, Memory, and Locality	Lab2 array traversal loop: <code>lw/addi</code> over a sequential array	The way data is traversed matters as much as the algorithm: sequential access is orders of magnitude faster than random access because hardware prefetches neighboring cache lines.
3	Why Your IoT Code Would Kill a Battery in Days: Polling, Interrupts, and Power	Lab3 busy-wait <code>sleep</code> loop; button-polling <code>ecall</code> loop	Polling loops burn far more power than necessary; a CPU can sleep and be woken by hardware events, and the <code>ecall</code> instruction students have been using is the same trap mechanism as a hardware interrupt.
4	The RISC-V Menu: Why Your Simulator Has “IM” in Its Name	Lab4 multiply via repeated addition (RV32I); <code>mul</code> instruction (M extension)	ISA extensions involve real costs (silicon area, verification complexity, compatibility risk); the “IM” designation reflects deliberate engineering tradeoffs, not a simple feature list.
5	The Hidden Cost of a Chain: Carry Propagation and the Subtraction Trick	Lab5 full adder and 4-bit ripple carry adder in Logisim	Chaining carry across bits creates a speed bottleneck that worsens with width; the same adder circuit handles subtraction without extra hardware via a single control wire that inverts inputs and supplies +1.
6	One ALU to Rule Them All? Why Processors Use Specialized Execution Units	Lab6 32-bit ALU from 1-bit cells; MUX-based operation selection via control signal	Floating point and multiplication require fundamentally different hardware from integer arithmetic, motivating specialized parallel execution units and introducing the intuition behind overlapping instruction execution.
7	When Programs Go Wrong: CPU Exceptions and Hardware Fault Handling	Lab7 single-cycle RISC-V CPU	Real processors handle instruction failures through a hardware mechanism that saves state and transfers control to a trusted handler; the privilege boundary is enforced in hardware, connecting back to the <code>ecall</code> instruction used throughout the course.

explores concepts that extend beyond what was covered in class, and this component asks students to close that loop by articulating how their new understanding connects back to the code or design decisions they already worked through in the lab.

To ground students in productive AI interaction before the first assignment, we provided an annotated example dialog on Dennard Scaling with explicit teaching notes identifying productive inquiry strategies: decomposing complex topics into focused sub-questions, requesting analogies, paraphrasing understanding back to the AI, following curiosity beyond the minimum, and refining rather than accepting first responses [23]. This expectation-setting step is a structural component of DGL; students are not expected to discover effective inquiry strategies independently.

All seven assignments were graded by teaching assistants under instructor oversight. TAs received onboarding on the DGL framework and its evaluative approach prior to the first assignment, with three check-in meetings across the term to address questions and calibrate on ambiguous cases²; TAs were also encouraged to escalate uncertain grading decisions to the instructor at any time.

C. Course Deployment and Assignment Topics

The DGL assignments were deployed in CS281: Systems Architecture, an undergraduate course at Drexel University.

²One submission included a leftover fragment of the student’s own prompt to the AI (“Do you want me to make this sound more like a confused student?”), illustrating that some students will attempt to game any format. TAs were instructed to flag such cases; the broader deterrent was reinforcing in class that critical AI engagement is a skill worth having on its own terms, and that convincingly faking the investigative process is roughly as time-consuming as doing it genuinely.

The course takes students from writing code to building the machine that runs it. The first half covers Instruction Set Architecture (ISA) concepts and RISC-V assembly programming using standard open source RISC-V developer tooling. The second half shifts to hardware, progressing through digital logic, ALU construction, and datapath design using Logisim circuit simulation [24]. In the final lab, students implement and simulate a working single-cycle RV32IM processor that executes machine code from programs they wrote in the first half. Enrollment was $N = 105$ students, drawn primarily from second- and third-year CS majors with limited prior hardware and systems exposure.

The course included seven DGL assignments, each paired with a major programming or design lab. Each assignment opened with a background section referencing specific code or circuits the student had just built, and targeted an investigation topic that extended the lab’s core concepts in a direction not addressed in lecture. Table II summarizes all seven assignments with their lab anchors and deliberate gaps.

D. Research Questions

This study addresses three primary research questions:

RQ1 What learning gains do students report across DGL assignments in an undergraduate systems architecture course, as measured by self-reported retrospective pre/post conceptual understanding ratings on each assignment’s learning objectives?

RQ2 Do students engage with AI in genuinely iterative, critical ways when required by assignment design to document their inquiry process, or do they default to surface-level, single-exchange interactions regardless of requirements?

RQ3 How do students perceive the DGL format relative to conventional homework in terms of learning value, cognitive engagement, and assignment clarity?

The DGL framework itself is a primary contribution: a transferable assignment design methodology including rubric structure, academic integrity approach, and the specific design choices that make genuine AI engagement necessary rather than optional. The empirical findings serve as evidence for the framework’s effectiveness and constraints.

IV. METHODOLOGY

The empirical component of this study relies on a longitudinal anonymous survey instrument administered alongside each DGL assignment. This section describes the survey design, the three categories of measures it captures, and the data collection approach.

A. Survey Instrument Design

A voluntary anonymous survey was administered after each DGL assignment via Qualtrics [25], a web-based survey platform. Participation was incentivized through a modest extra-credit opportunity: students who completed the survey received a unique receipt identifier which they submitted with their homework. Receipts were validated for authenticity without linking to individual responses, preserving anonymity while enabling completion verification.

The survey follows a consistent two-part structure across all assignments. The first part contains questions specific to the assignment’s learning objectives, varying from homework to homework. The second part contains a common battery of questions administered identically after every assignment, enabling longitudinal comparison. Each administration takes approximately five minutes to complete.

B. Survey Measures

The survey captures three categories of data. **Conceptual understanding** is measured in the assignment-specific section: for each of the five or six stated learning objectives, students rate their understanding on a five-point Likert scale both before and after the investigation, recalled retrospectively at survey time. Retrospective pre-testing mitigates the response-shift bias common in traditional pre/post designs, in which students over-rate prior knowledge before instruction, a distortion they can only recognize in retrospect [21], [22].

AI engagement quality is measured through questions characterizing the depth of the student’s AI dialogue. The primary engagement item asked students to select the description that best characterized their AI interaction during the assignment, from four options: (1) a single initial exchange with no meaningful follow-up; (2) back-and-forth dialogue with multiple follow-up questions; (3) active critical dialogue in which the student explicitly challenged, refined, or verified AI responses; or (4) used AI as a starting point and verified key claims with other sources. Additional items captured follow-up frequency and copy temptation on five-point Likert scales. Cross-validation against submitted Investigation Logs would

provide an observable behavioral check on these self-reports, though doing so requires non-anonymous student records and is addressed as a planned next step in Section VI.

Perceived learning value is measured via ratings of the overall value of the DGL format, a direct comparison to conventional homework, and identification of which of the four rubric sections students found most and least valuable. The Critical Check section is of particular interest; prior to deployment, the author expected students to rate it as least valuable, viewing it as “catching the AI” rather than as a meaningful skill. Section V shows this negative expectation was not supported, as the Critical Check was consistently among the more valued components, a finding that itself informed the design recommendations discussed later in the paper.

Survey links were distributed after each assignment was complete, ensuring that students had finished their investigation before responding. Because understanding is rated retrospectively, there is no risk that survey questions influenced how students approached the work. Data are anonymous with no personally identifying information gathered or linked to submissions. Responses were analyzed both within each assignment to examine per-objective gains, engagement, and value ratings, and across the full quarter to identify longitudinal trends. This study was conducted under Drexel University Institutional Review Board approval (Protocol #2605011810). The research was determined to be exempt research involving anonymous educational surveys with no collection of personally identifying information.

V. FINDINGS

A. Participation and Data Collection

Survey participation was incentivized through a modest 3% extra credit opportunity, consistent with other optional extra credit available throughout the course such as lab stretch objectives, and remained stable across the deployment. Table III summarizes response counts per assignment. HW1 achieved full participation; HW2–HW7 stabilized near 68–70% and did not continue to decline, indicating no ongoing attrition beyond the initial drop. The 70-respondent floor at HW7 is sufficient for the within-assignment analyses reported below.

B. RQ1: Self-Reported Learning Gains

1) *Normalized gain scores*: For each assignment, students retrospectively rated their prior conceptual understanding (pre) and their understanding after completing the investigation (post) on five-point Likert scales anchored to the assignment’s specific learning objectives. Following Hake’s normalized gain framework [20], we compute $g = (\text{post} - \text{pre}) / (5 - \text{pre})$ for each respondent. Cases where pre-score equals 5 are excluded as the gain formula is undefined at that value; no such cases occurred in our dataset. We adopt this metric as a descriptive effect measure applied to ordinal Likert responses rather than assuming interval-scale measurement, consistent with its increasing use in self-report learning contexts [21]. Scores are interpreted as low ($g < 0.30$), medium ($0.30 \leq g < 0.70$), or high ($g \geq 0.70$), following the thresholds established by

Hake [20] as broadly adopted reference points in the normalized gain literature, treated here as approximate benchmarks for ordering relative performance across assignments rather than as validated criteria for self-reported data.

Table IV reports per-assignment results. All seven assignments produced normalized gains in the medium-to-high range (lowest mean $g = 0.332$, HW7). Across the full deployment, 72% of individual gain scores fell in the medium or high category. Figure 1 visualizes the per-assignment gains and marks the point where the course shifted from ISA and assembly programming to digital hardware design. Paired-samples t -tests confirmed statistically significant pre-to-post gains on every assignment (t -values: 7.15–17.26, all $p < 0.001$), with large effect sizes (Cohen’s $d_z = 0.87$ –1.71 per assignment; pooled $d_z = 1.22$; computed as the mean difference divided by the standard deviation of the difference scores). Wilcoxon signed-rank tests on the ordinal Likert responses yielded consistent conclusions (all $|z| > 5.29$, $p < 0.001$). Effect sizes of this magnitude are characteristic of retrospective pretest designs, in which students rate prior knowledge after experiencing learning that recalibrates their baseline; they should be interpreted as indicators of perceived gain rather than absolute measures of knowledge change [21].

2) *An ISA/Software-to-hardware discontinuity*: A notable pattern emerges in the longitudinal data. HW1–4 (ISA/Software-level topics) produce average gains of $\bar{g} = 0.54$, while HW5–7 (circuit-level hardware) average $\bar{g} = 0.34$. This is not gradual drift: the drop between HW4 and HW5 is the sharpest single-step change in the dataset (0.49 to 0.35).

Three explanations are plausible and likely interact. First, the hardware assignments likely imposed greater cognitive demands: students were simultaneously building complex circuits in Logisim and investigating conceptual extensions, leaving less cognitive slack for investigation. Second, the deliberate gap is potentially wider in the hardware half: circuit-level topics are further from what CS students can ground in their everyday intuition. Third, student motivation and affect

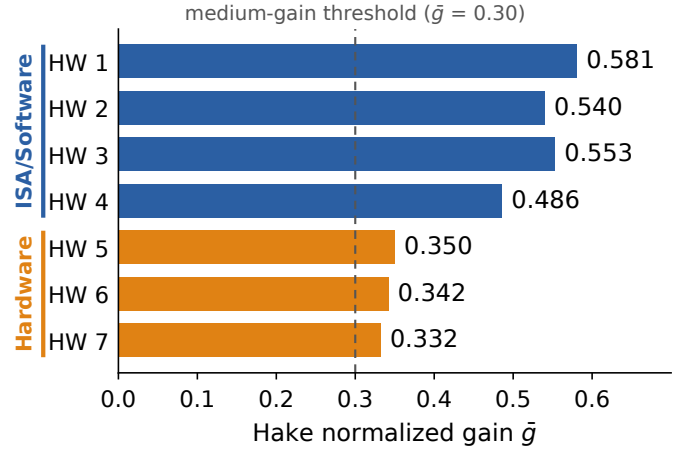


Fig. 1. Mean Hake normalized gain (g) per assignment. ISA/Software-level assignments (HW1–4, blue) average $\bar{g} = 0.54$; hardware-level assignments (HW5–7, amber) average $\bar{g} = 0.34$. Largest drop occurs between HW4 and HW5, coinciding with the course’s shift from software to circuit-level design.

likely play an independent role: CS students often find the shift to digital logic and processor design at mid-quarter less intrinsically motivating than assembly programming (see Section VI). A more speculative possibility is that the AI partner itself may be less effective in this domain: gate-level logic and circuit topology are areas where LLM reliability is not as established as linguistic or procedural reasoning, though this study has no direct measure of AI response quality to test this hypothesis. Lab-connection ratings decline from a consistent ≈ 3.8 in HW1–4 to 3.40–3.66 in HW5–7, aligning with all four explanations.

This discontinuity is a meaningful design finding: DGL appears more effective when students have an accessible conceptual foothold in the investigation topic and some prior interest or background in the domain.

C. RQ2: AI Engagement Quality

1) *Engagement mode*: After each assignment, students selected which of four options best described how they had primarily used the AI: asking questions and accepting the responses directly without follow-up (Mode 1); holding a back-and-forth conversation while mostly accepting what the AI said (Mode 2); actively pushing back, asking follow-ups, and questioning answers (Mode 3); or using the AI as a starting point and verifying key claims with other sources (Mode 4). Table V reports mode distributions. Figure 2 visualizes them as stacked bars.

The data offer a clear answer to RQ2: the large majority of students consistently engaged with AI in back-and-forth, active critical, or cross-checking modes. The deep engagement rate held between 87% and 96% across all seven assignments, including the final three where overall satisfaction was declining. This stability is notable: even as students expressed fatigue with the format, they continued to engage substantively rather than defaulting to shortcut behavior. These engagement characterizations are self-reported; cross-validation against submitted investigation logs is a planned next step pending

TABLE III
SURVEY PARTICIPATION BY ASSIGNMENT ($N=105$ ENROLLED); TOTAL RESPONSES: 538; MEAN PARTICIPATION RATE: 73.2%.

HW	Topic	Responses	Participation Rate
1	x86 instruction complexity, micro-ops, and Dennard Scaling	105	100.0%
2	Cache hierarchy, spatial locality, and sequential vs. random access	74	70.5%
3	Polling, hardware interrupts, and the trap mechanism	72	68.6%
4	ISA extension tradeoffs: silicon area, compatibility, and the M extension	71	67.6%
5	Ripple-carry bottleneck, adder width, and two’s complement subtraction	72	68.6%
6	Floating-point hardware, execution unit specialization, and pipelining	74	70.5%
7	Exception handling, privilege boundaries, and hardware fault isolation	70	66.7%

TABLE IV

PER-ASSIGNMENT NORMALIZED LEARNING GAINS AND LAB-CONNECTION RATINGS. $g = (\text{POST} - \text{PRE}) / (5 - \text{PRE})$, EXCLUDING PRE = 5 CASES. 95% CIs COMPUTED AS $1.96 \times \text{SE}$. LOW: $g < 0.30$; MEDIUM: $0.30 \leq g < 0.70$; HIGH: $g \geq 0.70$.

HW	Topic	<i>n</i>	Pre	Post	$\bar{g}$	95% CI	Low	Med	High	Lab Link
1	CISC/RISC complexity & power	102	1.94	3.82	0.581	± 0.058	13%	50%	37%	3.77
2	Cache hierarchy & memory locality	69	2.66	3.99	0.540	± 0.106	22%	46%	32%	3.85
3	Interrupts & IoT power	72	1.97	3.83	0.553	± 0.086	21%	33%	46%	3.74
4	RISC-V ISA extensions	69	2.51	3.90	0.486	± 0.087	26%	46%	28%	3.79
5	Carry propagation & subtraction	67	2.65	3.61	0.350	± 0.103	40%	43%	16%	3.66
6	Specialized execution units	69	2.44	3.38	0.342	± 0.074	41%	49%	10%	3.40
7	CPU exceptions & privilege modes	66	2.46	3.44	0.332	± 0.075	44%	45%	11%	3.52
Pooled		514	2.38	3.71	0.464		28%	45%	27%	3.68

TABLE V

AI ENGAGEMENT MODE DISTRIBUTION BY ASSIGNMENT. DEEP ENGAGEMENT = MODES 2+3+4 COMBINED.

HW	Mode 1: Accept & move on	Mode 2: Back-and- forth	Mode 3: Active critical	Mode 4: Cross- checked	Deep (2+3+4)
1	4%	46%	44%	7%	96.0%
2	8%	34%	52%	6%	91.5%
3	7%	54%	29%	10%	93.1%
4	7%	46%	45%	1%	93.0%
5	6%	47%	44%	3%	94.3%
6	12%	49%	36%	3%	87.5%
7	12%	47%	40%	1%	88.2%
Avg	8%	46%	41%	4%	92.2%

a protocol amendment covering behavioral analysis of student work products (see Section VI).

2) *Follow-up frequency and copy temptation*: Follow-up frequency averaged 3.14/5 across all HWs, indicating students were frequently probing for clarification. Copy temptation averaged 2.25/5, indicating low temptation. Copy temptation ranged from only 2.15 (HW1) to 2.40 (HW3) with no upward trend across the quarter, directly rebutting the concern that students would increasingly exploit the format as they grew familiar with it.

3) *Engagement mode and learning gain*: Pooling across all seven assignments, students in Modes 2, 3, and 4 showed substantially higher average Hake gains than Mode 1 students ($\bar{g} = 0.46, 0.50, \text{ and } 0.59$ vs. 0.26). Welch independent-samples *t*-tests were used to account for unequal group variances; degrees of freedom are Welch-Satterthwaite estimates and thus non-integer before rounding. These differences were statistically significant (Mode 2 vs. Mode 1: $t(45) = 2.31$, $p < 0.05$, $d = 0.53$; Mode 3 vs. Mode 1: $t(46) = 2.70$, $p < 0.01$, $d = 0.60$; Mode 4 vs. Mode 1: $t(55) = 2.73$, $p < 0.01$, $d = 0.67$). These Cohen's *d* values reflect between-group effect sizes and are distinct from the paired d_z reported for pre/post gains. Students who used AI as a starting point and verified claims with other sources (Mode 4) reported the highest average gain of the four modes. This within-condition pattern ($\Delta g \approx 0.20\text{--}0.33$ relative to Mode 1) is the strongest internal evidence for an engagement mechanism and is difficult to attribute to course content alone, though causal direction

cannot be established from self-report data.

D. RQ3: Perceived Learning Value

1) *Overall ratings and format comparison*: All perception items use a five-point scale anchored at 3 (neutral or equivalent); scores above 3 indicate a positive response and scores below 3 indicate a negative one. Table VI reports the three summary perception items across all assignments. Figure 3 plots the longitudinal trends.

Overall ratings declined from 3.49 (HW1) to 3.15 (HW7) and the format comparison item ended at 3.04, essentially neutral relative to conventional homework. No metric collapsed, but the decline is real and tracks consistently across all four perception items.

2) *Learning feel and format fatigue*: The learning-feel item declined from 2.92 to 2.57 across the quarter; by HW7, 40% of students rated the experience as primarily task-like vs. 19% as primarily learning.

Gain scores indicate substantial self-reported learning, but the subjective experience of that learning is not consistently what students would classify as genuine intellectual engagement. The writing burden, the structured section format, and the repetition across seven assignments all appear to compound into a task-completion frame by the second half of the quarter, even when the underlying conceptual work remains substantive.

3) *Critical Check reception*: Critical Check ratings averaged 3.09–3.32/5, above the scale midpoint on every assignment, consistently the second-highest rated component behind the Synthesis section, directly contradicting our pre-deployment hypothesis that students would view it as least

TABLE VI

LONGITUDINAL PERCEPTION RATINGS (ALL ITEMS 1–5; MIDPOINT 3 = NEUTRAL). FOR AI vs. *trad.*: 3 = EQUIVALENT TO TRADITIONAL, >3 = DGL RATED BETTER.

HW	Overall	AI vs. trad.	Want more	Learn feel
1	3.49	3.26	3.27	2.92
2	3.56	3.49	3.38	3.07
3	3.42	3.19	3.11	3.03
4	3.34	3.11	3.17	2.80
5	3.26	3.21	3.11	2.73
6	3.21	3.06	3.15	2.68
7	3.15	3.04	2.99	2.57

valuable. A recurring concern in open responses involved students with limited topic background finding it difficult to identify AI errors, pointing to a genuine design tension: the Critical Check requires epistemic confidence to challenge an authoritative-sounding source, which may not develop until students have internalized the material.

4) *Qualitative themes from open responses*: Open responses across all seven assignments (538 total) were analyzed inductively by the author using thematic analysis. Initial open coding grouped comments into provisional categories; responses were re-read iteratively until no new categories emerged after HW4; no additional themes arose in HW5–7. Four stable patterns emerged, each appearing independently in at least five of the seven assignments. (1) **Genuine surprise at content depth**: students frequently reported unexpected conceptual connections: *I was surprised by how applicable caching is to logic I write in code. Before the assignment, I knew about caching but thought it was primarily out of reach to direct its effect, but now I clearly see locality in code and how it could impact things*” (HW2). (2) **AI as tutor unlocking follow-up curiosity**: *I got into a discussion with the AI about multi-core processors and how interruptions can be handled across cores; I thought that strategy was super interesting*” (HW3). (3) **Writing burden and structural repetition**: the most frequent criticism, with students calling out the investigation log length and the sameness of format across the quarter: *4 sections took too long, and most of the time wasn’t learning; it was just writing*” (HW1). (4) **Mandatory AI friction**: a consistent minority expressed resistance to required AI use; these responses grew louder in HW5–7 and are discussed further in Section VI: *The topic itself was interesting, but the mandatory use of AI made it a chore*” (HW7).

VI. DISCUSSION

A. What the Findings Mean for DGL as a Framework

Within the limits of this design, gains were consistent. All seven assignments produced normalized learning gains in the

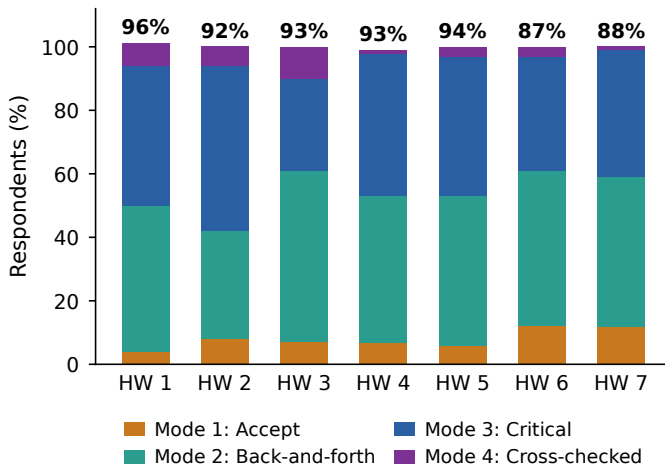


Fig. 2. AI engagement mode distribution per assignment. Bold values above bars show the combined deep-engagement rate (Modes 2+3+4), which held between 87% and 96% across all seven assignments.

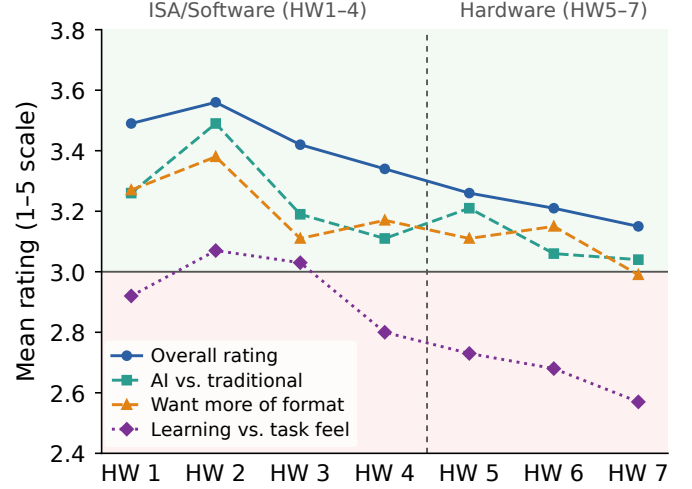


Fig. 3. Longitudinal perception ratings. All items use a 1–5 scale; green shading marks ratings above the neutral midpoint (3.0), red below. For *AI vs. traditional*, 3 = equivalent to conventional homework.

medium or better range ($\bar{g} = 0.33$ – 0.58) while students were completing DGL assignments. Without a control condition, these gains cannot be solely attributed to DGL. What the data *can* support is that higher engagement quality was associated with higher reported gains (Mode 1 $\bar{g} = 0.26$ vs. Mode 3 $\bar{g} = 0.50$), and that deep engagement held at 87–96% across all seven assignments. An instructor considering DGL should weight this accordingly: the observed self-reported gains appear consistent even when students do not find the format subjectively rewarding, and no increase in reported low-effort engagement was observed even by the final assignment.

The honest qualification is equally clear. By the second half of the course, students were more likely to experience DGL assignments as task completion than as genuine learning, and overall ratings declined consistently if modestly. Whether this reflects fatigue with the format, increased cognitive load from the hardware topics, or the compounding weight of seven structurally identical assignments is not separable from this dataset. All three explanations point toward the same design implication: the seven-assignment deployment volume was likely too high for sustained perceived engagement, and future iterations should vary the structure or alternate DGL with other formats.

The ISA/Software-to-hardware discontinuity in self-reported gains is the finding most directly actionable for course design. DGL works best when the deliberate gap is genuinely adjacent to what students already know. When both the lab and the gap are simultaneously challenging, the gain mechanism weakens.

B. Contextual Factors

1) *The systems architecture course context*: Computer organization and architecture is a universally required course in ACM-recommended CS curricula [26], but it covers topics (processor design, memory hierarchies, digital logic) that are far from what motivates most students to pursue CS. Both the instructor-side challenges of teaching this material [27]

and its difficulty for students [28] are well documented. This matters for interpreting the RQ3 results: a meaningful share of the declining “learning feel” scores may reflect baseline disengagement with the course topic rather than a specific response to DGL, and the near-parity of DGL in format comparison ratings is notable given that baseline.

2) *Unexpected student reception in an AI-fluency context:*

One of the more counterintuitive findings of this deployment concerns the gap between observed learning outcomes and student reception. Given the substantial professional pressure CS students currently face to demonstrate AI fluency (workforce projections consistently rank AI and data skills among the fastest-growing technical competencies [3]), the author anticipated that students would experience DGL as directly relevant career-skill practice and respond with correspondingly high engagement ratings. The data did not support this expectation: overall ratings peaked at 3.56 and ended near 3.15, and the format comparison item barely favored DGL over conventional homework at any point in the quarter.

A plausible explanation is that students distinguish between two different uses of AI: using AI to produce deliverables efficiently (the kind of productivity the market highlights) and using AI to investigate unfamiliar topics in structured, effortful ways (what DGL requires). DGL requires the latter, which is arguably the more professionally durable skill, but also the more cognitively demanding one. Per-assignment effort likely ranged from two to four hours, and using AI as a genuine learning partner (building understanding incrementally, requesting analogies, refining rather than accepting first explanations) describes cognitively active behaviors that differ meaningfully from the prompt-and-accept pattern most students have developed elsewhere. Students who expected AI to make homework *easier* may have found instead that DGL made it *differently demanding*.

3) *A hypothesis about the “mandatory AI” complaint:* A consistent minority expressed resistance to required AI use, explicitly requesting a return to traditional homework formats. One hypothesis is that this reflects an effort differential: a traditional written homework on cache hierarchy and memory locality would be straightforwardly solvable in minutes by any competent LLM. On this reading, the complaint about “mandatory AI” may in some cases be a complaint about being required to use AI *non-trivially*, when using AI trivially (to complete a traditional homework) would be faster and less cognitively demanding.

We flag this as a hypothesis, not a conclusion. Students who prefer traditional homework may do so for principled pedagogical reasons independent of effort calculation, and the data cannot distinguish between these explanations. Future deployments should acknowledge both possibilities explicitly rather than leaving students to discover the effort-alignment dynamic through experience.

C. Implications for Practice

Instructors considering DGL in similar lab-intensive courses should take four practical lessons from this deployment. First, limit initial deployment to four or five assignments per term

rather than the full seven used here; the engagement data suggest the mechanism is robust, but the perception data suggest seven identical structures compound into format fatigue. Second, vary the format in the second half of the course: a shorter or differently structured DGL variant for the hardware assignments, where cognitive load is highest, may preserve the gain mechanism while reducing the task-like framing that emerged in HW5–7. Third, address the effort-expectations gap explicitly before the first assignment: students who expect AI to make homework easier may disengage when they discover it redirects rather than reduces effort, and the tutorial framing should make this explicit from the outset. Fourth, invest in grader calibration before the term begins: DGL rubrics assess the quality and authenticity of an investigative process rather than the correctness of an answer, a skill that is not native to most instructors’ or teaching assistants’ grading experience. In this deployment, teaching assistants who performed all grading received upfront training on the DGL framework and its evaluative approach, with three check-in meetings across the term to calibrate on ambiguous cases; TAs reported that this shift added grading time but also found the evaluative process itself engaging as CS students in their own right.

D. Threats to Validity

Self-report validity and social desirability. All outcome measures are self-reported and are not equivalent to demonstrated mastery. Students who knew their engagement would be assessed may have over-reported depth. Additionally, the memory-based Synthesis requirement, designed to prevent direct AI copy-paste, cannot rule out a student paraphrasing AI output using a second tool; this behavioral variable is unobservable under the current anonymous survey protocol. Cross-validation against submitted investigation logs would directly address both concerns and remains the most important planned next step.

As informal corroboration, the author reviewed 25–50 submissions per assignment as part of normal course oversight; it was generally evident which students had engaged substantively and which had approached the assignment as a task to complete. This observation is reported only as informal corroboration, not as a statistical or scientific claim: the scope of the current IRB protocol (Protocol #2605011810; Category 2(i) exempt) was deliberately bounded to anonymous survey data, and a systematic comparison against submission-level records would require a materially more complex protocol covering non-anonymized academic records. That follow-on study, which would formally cross-validate self-reported engagement against submitted investigation logs, is the immediate next step.

Voluntary participation. Approximately 30% of students are not represented in HW2–7 data; disengaged students may be systematically more likely to skip the survey, potentially biasing ratings upward.

Common-method variance. Because conceptual gains, engagement quality, and perceived learning value were all collected through the same survey instrument administered at the same time, shared-method variance may inflate observed

associations among these measures. The within-survey correlations between engagement mode and reported gain should therefore be interpreted with caution.

Absence of a control condition. The gains reported here cannot be attributed to DGL specifically versus course instruction, labs, general exposure, or time on task. The honest claim is that students reported gains *while using* DGL and that higher engagement mode was associated with higher reported gains within the DGL condition. Separating these factors would require a more controlled comparison design; this baseline characterization was intentionally scoped to minimize participant burden and administrative overhead so that initial evidence could be gathered before committing to a more resource-intensive protocol.

Single course, single instructor, single institution; AI model variability. Transfer to other courses requires replication studies. Students used a variety of LLMs, and model-specific effects on learning outcomes are not captured.

Multiple comparisons. Separate statistical tests were run for each of the seven assignments rather than across the dataset as a whole. To guard against inflated Type I error from running seven tests, we apply a Bonferroni-corrected threshold of $\alpha = 0.05/7 \approx 0.0071$; every assignment’s pre/post gain remains significant at this stricter threshold ($p < 0.001$ in all cases), so the multiple-comparisons concern does not change any conclusion in this paper.

VII. CONCLUSION

This paper presents a first empirical characterization of Deliberate Gap Learning as a baseline for a research program on AI-integrated homework design. The central contribution is a redistribution of instructional roles: lecture establishes conceptual foundations, labs develop procedural competence, and homework extends understanding into territory intentionally left uncovered, using AI as the inquiry partner. By grading process rather than answers, DGL is designed to substantially reduce the value of low-effort AI answer generation while opening a high-value path that is transferable across tool generations and requires no custom infrastructure.

The seven-assignment deployment in CS281 supports the framework’s core claims. Students reported consistent medium-to-high self-reported conceptual gains ($\bar{g} = 0.46$ pooled), engaged deeply and non-exploitatively with AI throughout the quarter (92.2% average deep engagement across assignments; copy temptation stable at $\approx 2.25/5$), and rated the critical evaluation component more highly than pre-deployment expectations. These results held even as format satisfaction declined across the quarter, suggesting that reported gains are partially decoupled from the student’s subjective experience of the assignment.

The study also surfaces important limitations and unanticipated findings. The ISA/Software-to-hardware discontinuity in gains points to a design constraint: DGL is most effective when the deliberate gap is genuinely adjacent to what students know. The consistent decline in perceived learning value across seven structurally identical assignments suggests that deployment volume and format variety need attention. And despite a job

market that rewards AI fluency, student reception was warmer than neutral but not enthusiastic, a finding that warrants explicit attention to how DGL assignments are framed relative to professional AI practice.

The findings reported here establish that baseline: DGL is deployable without custom infrastructure, was associated with consistently large self-reported gains under genuine engagement, and showed no increase in reported low-effort engagement when rubric design required process documentation. They also reveal something more interesting than simple success or failure: when students are required to use AI non-trivially for learning rather than for output production, they encounter a cognitive challenge many did not anticipate. That challenge, and how assignment design, expectation-setting, and deployment volume interact to shape students’ experience of it, is the research agenda this study opens.

The baseline findings motivate four specific next steps: cross-validation of self-reported engagement with submitted investigation logs; a comparison condition to isolate the DGL effect; replication outside systems architecture; and systematic manipulation of format variety to separate effort from repetition as sources of fatigue. The framework, rubric, and example assignments used in this study are publicly available [23].

ACKNOWLEDGMENT

The author thanks the CS281 students at Drexel University whose participation and survey responses made this study possible.

REFERENCES

- [1] D. Weber-Wulff, A. Anohina-Naumeca, S. Bjelobaba, T. Foltýnek, J. Guerrero-Dib, O. Popoola, P. Šigut, and L. Waddington, “Testing of detection tools for AI-generated text,” *International Journal for Educational Integrity*, vol. 19, no. 1, p. 26, 2023.
- [2] W. Liang, M. Yuksekogonul, Y. Mao, E. Wu, and J. Zou, “GPT detectors are biased against non-native English writers,” *Patterns*, vol. 4, no. 7, p. 100779, 2023.
- [3] World Economic Forum, “Future of jobs report 2025,” World Economic Forum, Tech. Rep., 2025, retrieved June 2026. [Online]. Available: <https://www.weforum.org/reports/the-future-of-jobs-report-2025/>
- [4] M. J. Prince and R. M. Felder, “Inductive teaching and learning methods: Definitions, comparisons, and research bases,” *Journal of Engineering Education*, vol. 95, no. 2, pp. 123–138, 2006.
- [5] S. Freeman, S. L. Eddy, M. McDonough, M. K. Smith, N. Okoroafor, H. Jordt, and M. P. Wenderoth, “Active learning increases student performance in science, engineering, and mathematics,” *Proceedings of the National Academy of Sciences*, vol. 111, no. 23, pp. 8410–8415, 2014.
- [6] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather, “The robots are coming: Exploring the implications of OpenAI Codex on introductory programming,” in *Proceedings of the 24th Australasian Computing Education Conference (ACE ’22)*. ACM, 2022, pp. 10–19.
- [7] B. A. Becker, P. Denny, J. Finnie-Ansley, A. Luxton-Reilly, J. Prather, and E. A. Santos, “Programming is hard—or at least it used to be: Educational opportunities and challenges of AI code generation,” in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE ’23)*. ACM, 2023, pp. 500–506.
- [8] J. Prather, B. N. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, and E. A. Santos, “It’s Weird That It Knows What I Want: Usability and interactions with Copilot for novice programmers,” *ACM Transactions on Computer-Human Interaction*, vol. 31, no. 1, 2023.

- [9] J. Prather, B. N. Reeves, J. Leinonen, S. MacNeil, A. S. Randrianasolo, B. A. Becker, B. Kimmel, J. Wright, and B. Briggs, "The widening gap: The benefits and harms of generative AI for novice programmers," in *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1*, ser. ICER '24. New York, NY, USA: ACM, 2024.
- [10] M. Perkins, "Academic integrity considerations of AI large language models in the post-pandemic era: ChatGPT and beyond," *Journal of University Teaching & Learning Practice*, vol. 20, no. 2, 2023.
- [11] D. R. E. Cotton, P. A. Cotton, and J. R. Shipway, "Chatting and cheating: Ensuring academic integrity in the era of ChatGPT," *Innovations in Education and Teaching International*, vol. 61, no. 2, pp. 228–239, 2023.
- [12] E. R. Mollick and L. Mollick, "Assigning AI: Seven approaches for students, with prompts," 2023. [Online]. Available: <https://arxiv.org/abs/2306.10052>
- [13] P. Denny, J. Prather, B. A. Becker, J. Finnie-Ansley, A. Hellas, J. Leinonen, A. Luxton-Reilly, B. N. Reeves, E. A. Santos, and S. Sarsa, "Computing education in the era of generative AI," *Communications of the ACM*, vol. 67, no. 2, pp. 56–67, 2024.
- [14] M. Liffiton, B. E. Sheese, J. Savelka, and P. Denny, "CodeHelp: Using large language models with guardrails for scalable support in programming classes," in *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. New York, NY, USA: ACM, 2023.
- [15] A. Vadaparty, D. Zingaro, D. H. Smith IV, M. Padala, C. Alvarado, J. G. Benario, and L. Porter, "CS1-LLM: Integrating LLMs into CS1 instruction," in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*. New York, NY, USA: ACM, 2024.
- [16] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, "Studying the effect of AI code generators on supporting novice learners in introductory programming," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. New York, NY, USA: ACM, 2023.
- [17] D.-K. Kim and M. Hamdi, "Evaluating the effectiveness of AI integration in software engineering education," *ACM Transactions on Computing Education*, vol. 26, no. 4, May 2026.
- [18] D. Long and B. Magerko, "What is AI literacy? Competencies and design considerations," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 2020, pp. 1–16.
- [19] D. T. K. Ng, J. K. L. Leung, S. K. W. Chu, and M. S. Qiao, "Conceptualizing AI literacy: An exploratory review," *Computers and Education: Artificial Intelligence*, vol. 2, p. 100041, 2021.
- [20] R. R. Hake, "Interactive-engagement versus traditional methods: A six-thousand-student survey of mechanics test data for introductory physics courses," *American Journal of Physics*, vol. 66, no. 1, pp. 64–74, 1998.
- [21] G. S. Howard and P. R. Dailey, "Response-shift bias: A source of contamination of self-report measures," *Journal of Applied Psychology*, vol. 64, no. 2, pp. 144–150, 1979.
- [22] M. Sprangers and J. Hoogstraten, "Pretesting effects in retrospective pretest-posttest designs," *Journal of Applied Psychology*, vol. 74, no. 2, pp. 265–272, 1989.
- [23] B. Mitchell, "Online dgl paper resources," GitHub Repository, May 2026. [Online]. Available: <https://github.com/drexel-cn-research/cnse-public/tree/main/DGL>
- [24] LogiSim Evolution Team. (2026) Logisim Evolution - Digital logic design tool and simulator. [Online]. Available: <https://github.com/logisim-evolution/logisim-evolution>
- [25] Qualtrics, "Qualtrics," Qualtrics International Inc., Provo, UT, 2024. [Online]. Available: <https://www.qualtrics.com>
- [26] ACM/IEEE-CS Joint Task Force on Computing Curricula, "Computer science curricula 2023," 2023, retrieved June 2026. [Online]. Available: <https://www.acm.org/education/curricula-recommendations>
- [27] L. Cassel, D. Kumar, K. Bolding, J. Davies, M. Holliday, J. Impagliazzo, G. Wolffe, and W. Yurcik, "Distributed expertise for teaching computer organization and architecture," *ACM SIGCSE Bulletin*, vol. 33, no. 2, pp. 111–126, 2001.
- [28] C. Yehezkel, M. Ben-Ari, and T. Dreyfus, "The contribution of visualization to learning computer architecture," *Computer Science Education*, vol. 17, no. 2, pp. 117–127, 2007.